



Week 2: Algorithm Analysis



Dr. Sirasit Lochanachit



HOW WOULD YOU TRAVEL FROM BANGKOK TO CHIANG MAI?

TRANSPORTATION AS ALGORITHMS: TIME COMPLEXITY (Bangkok to Chiang Mai)



Algorithm		Time Complexity (Estimated Time)
	Algorithm 1: Walking	Very High (~140-160 hours)
	Algorithm 2: Bus/Train	High (~10-13 hours)
	Algorithm 3: Driving (Car)	Medium (~8-10 hours)
	Algorithm 4: Flying (Plane)	Low (~1-1.5 hours)

Different algorithms (transport methods) solve the same problem (reaching the destination) but with varying time complexities.

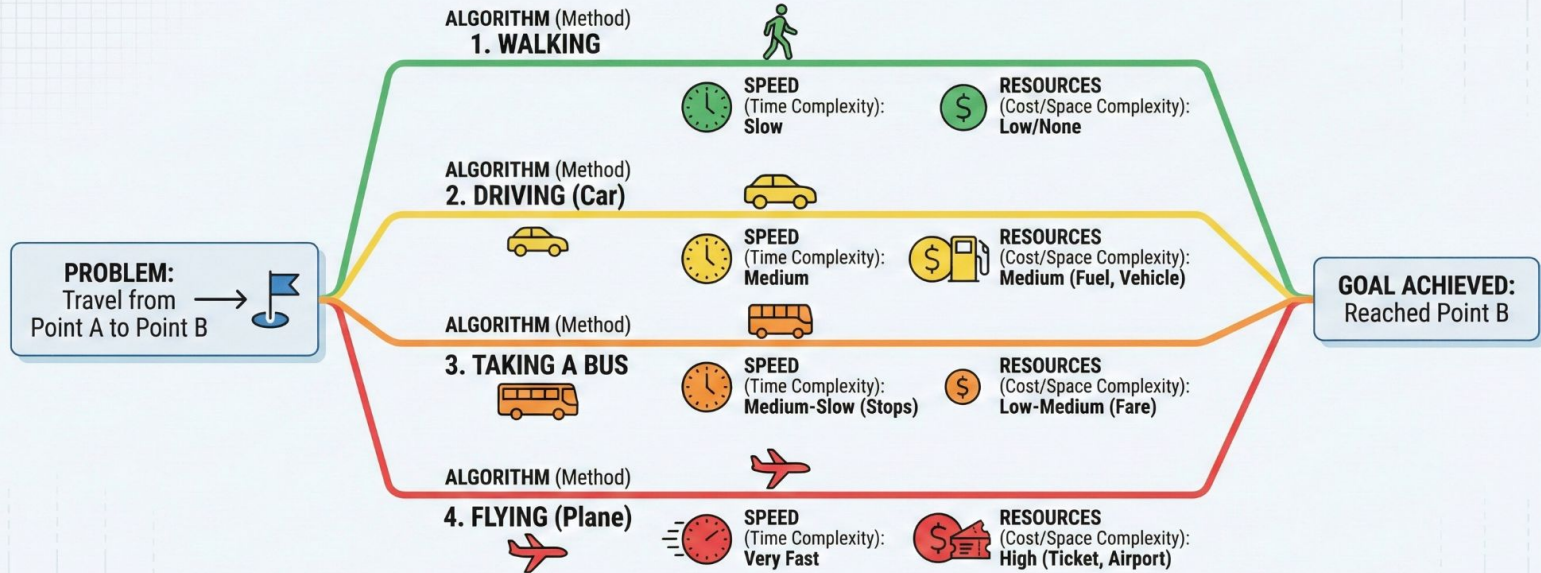
TRANSPORTATION AS ALGORITHMS: COST/SPACE COMPLEXITY (Bangkok to Chiang Mai)



Algorithm		Cost/Space Complexity (Estimated Cost)
	Algorithm 1: Walking	Very Low (Food/Water); Low Space
	Algorithm 2: Bus/Train	Low (~500-1500 THB); Medium Space (Seat/Luggage)
	Algorithm 3: Driving (Car)	Medium (~2000-4000 THB, Fuel/Rental); Medium Space (Car/Trunk)
	Algorithm 4: Flying (Plane)	High (~1500-5000+ THB); High Space (Airport/Seat/Baggage fees)

Different algorithms (transport methods) solve the same problem (reaching the destination) but with varying cost and space complexities.

ONE PROBLEM, MULTIPLE ALGORITHMS



KEY TAKEAWAY: Different algorithms solve the same problem but have different trade-offs in Time and Space (Cost) complexity. Choosing the right one depends on constraints.



Computational Complexity



- If computers were infinitely fast, **any correct method** for solving a problem would do.
 - In the real world, computing time and space in memory are a limited resource.
- Algorithms that are efficient in terms of **time or space** are preferred.



Computational Complexity



- How do we measure algorithm computational efficiency?
- How do we measure algorithm's resources usage?
- How do we measure algorithm's **efficiency** or **performance**?
 - I.e. How to benchmark any algorithm?



Measuring Algorithms Performance



1. Use **running time** as an indicator.



Example - Summation of n integers

```
def sumOfN(n:int):
```

```
    total = 0
```

```
    for i in range(1, n+1):
```

```
        total = total + i
```

```
    return total
```



Running Time of Summation



- For-loop version

Time

sumOfN(10000)

sumOfN(100000)

sumOfN(1000000)

Which run should be slowest?



Example - Summation of n integers

```
def sumOfN(n:int):
```

```
    total = 0
```

```
    for i in range(1, n+1):
```

```
        total = total + i
```

```
    return total
```

- Time required seems to increase as we increase the input size (n).



Running Time of Summation



- For-loop version

Time

sumOfN(10000)

sumOfN(100000)

sumOfN(1000000)

- Formula based version

Time

sumOfN2(10000)

sumOfN2(100000)

sumOfN2(1000000)

Is formula based version faster than for loop version?



Running Time Factors



- Input size (how it scales)



Example: Primality Testing



1. Magnitude as Complexity

- The *magnitude of the input value* (e.g., for numeric input)

Example: When check if a number n is a prime, we loop up to $\text{sqrt}(n)$.

- If $n = 100$, loop runs ? times
- If $n = 10,000,000$, loop runs ? times



Example: Primality Testing



2. Number of Items as Complexity

- The number of items in the input (e.g., as in a list)

Example: Suppose a teacher has a list of n student names, searching for a specific name requires checking one by one.

- **Small** input: $n = 10$ items. Almost instant result.
- **Large** input: $n = 1,000,000$ items. Requires significant time.

Complexity is proportional to the **count** of items



Input Size Considerations



- *Input size* might be:
 - the *magnitude of the input value* (e.g., for numeric input)
 - the *number of items* in the input (e.g., as in a list)
- An algorithm may also be dependent on *more than one input*.



Running Time Factors

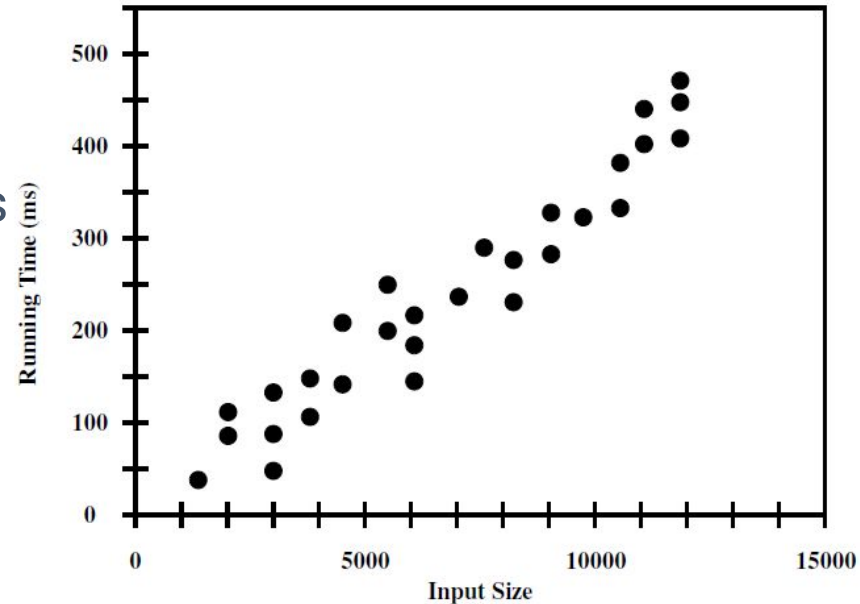


- Input size (how it scales)
- Running time also depends on many factors
 - Hardware (CPU, RAM, etc.)
 - Software (OS, Programming language, etc.)

Experimental Studies

To measure how the algorithm scales (i.e. input size increases)

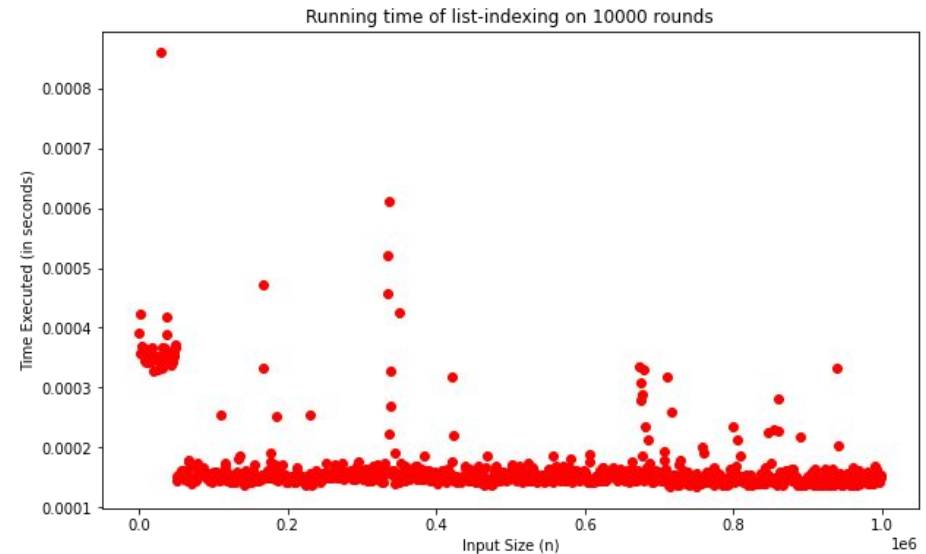
- Implement an algorithm then study its running time by
 - Executing with different test inputs of various sizes and
 - Recording time spent for each input size
 - Plot the results



Experimental Studies



Lab 3-2 and Lab 3-3 will explore this in more detail





Challenges of Experiments



- To directly compare between two different algorithms, the same hardware and software environments must be used.
- Limited set of test inputs.
- An algorithm must be fully implemented to study its running time.
 - Can be **costly** for some algorithms



High-level Analysis



- Instead of implementing an algorithm and perform experiments, we can study a high-level description of the algorithm.
 - Either in the form of an actual code or pseudo-code.
- Takes into account all possible inputs
- Allows us to evaluate the efficiency of an algorithm independent of hardware & software environment.



Computational Complexity



1. Use **running time** as an indicator.
2. Use **number of operations or steps** as an indicator.



Counting Primitive Operations



- Formally, a primitive operation corresponds to a low-level instruction with an execution time that is constant.
 - Assign a variable to an object
 - Determining the object associated with a variable
 - Performing an arithmetic operation
 - Comparing two numbers
 - Accessing a single element of a Python list by an index
 - Calling a function (excluding operations within the function)
 - Returning from a function



Counting Operations



```
def sumOfN(n:int):
```

```
    total = 0
```

```
    for i in range(1, n+1):
```

```
        total = total + i
```

```
    return total
```

Number of Operations

Time complexity

$T(n) =$



Seven Important Functions



- Seven fundamental functions in algorithm analysis:

- Constant: $f(n) = c$
- Logarithmic: $f(n) = \log_b n$
- Linear: $f(n) = n$
- N-Log-n: $f(n) = n \log n$
- Quadratic: $f(n) = n^2$
- Cubic: $f(n) = n^3$
- Exponential: $f(n) = 2^n$



Time Complexity



- To see long-term / big picture trends of running time
 - Given an algorithm that takes *input size* n , find a function $T(n)$ that describes the *runtime* of the algorithm

Loops

Python Code

Time complexity

Linear

```
for i in range(0, n):  
    do something
```



$T(n) =$

Linear

```
for i in range(0, n, 2):  
    do something
```



$T(n) =$

Nested

```
for i in range(0, n):  
    for j in range(0, n):  
        do something
```



$T(n) =$

A log is a Repeated Division

Python Code

Logarithmic

```

i = 1          Start
while i < n:   Stop
    do something
    i = i * 2   Step
    
```

$n=1000$

1
 2
 4
 8
 16
 32
 64
 128
 256
 512

Time complexity

$T(n) =$

Logarithmic

```

i = n          Stop
while i >= 1:  Start
    do something
    i = i // 2  Step
    
```

1000
 500
 250
 125
 62
 31
 15
 7
 3
 1

$T(n) =$



Linear Logarithmic Loops



Python Code

Time complexity

Linear
logarithmic

```
for i in range(0,n): n  
    j = 1  
    while j < n: log n  
        do something  
        j = j*2
```



$$T(n) =$$



Exponential



Python Code

Time complexity

Double Recursive

```
def foo(n):  
    if (n==1):  
        Return True  
  
    foo(n-1)  
    foo(n-1)
```



$$T(n) =$$



Computational Complexity



1. Use **running time** as an indicator.
2. Use **number of operations or steps** as an indicator.
 - Represented by $T(n)$

Do we need to count a precise number of operations?



Asymptotic Notation



$$T(n) = 2n^2 + n + 1$$

- The running time of this algorithm grows as n^2 .
- $T(n) \approx 2n^2 \approx n^2$
- $T(n)$ is asymptotic to n^2
 - When n approaches to infinity, the rests are **insignificant**.



Asymptotic Notation



- Asymptotic notation represents algorithm's complexity.
 - Ignores constant factors and slower growing terms.
 - Focus on the main components that affect the growth.
 - **Big-O notation**



Computational Complexity



1. Use **running time** as an indicator.
2. Use **number of operations or steps** as an indicator.
 - Represented by $T(n)$
3. Use asymptotic notation as an indicator.
 - **Big-O notation**

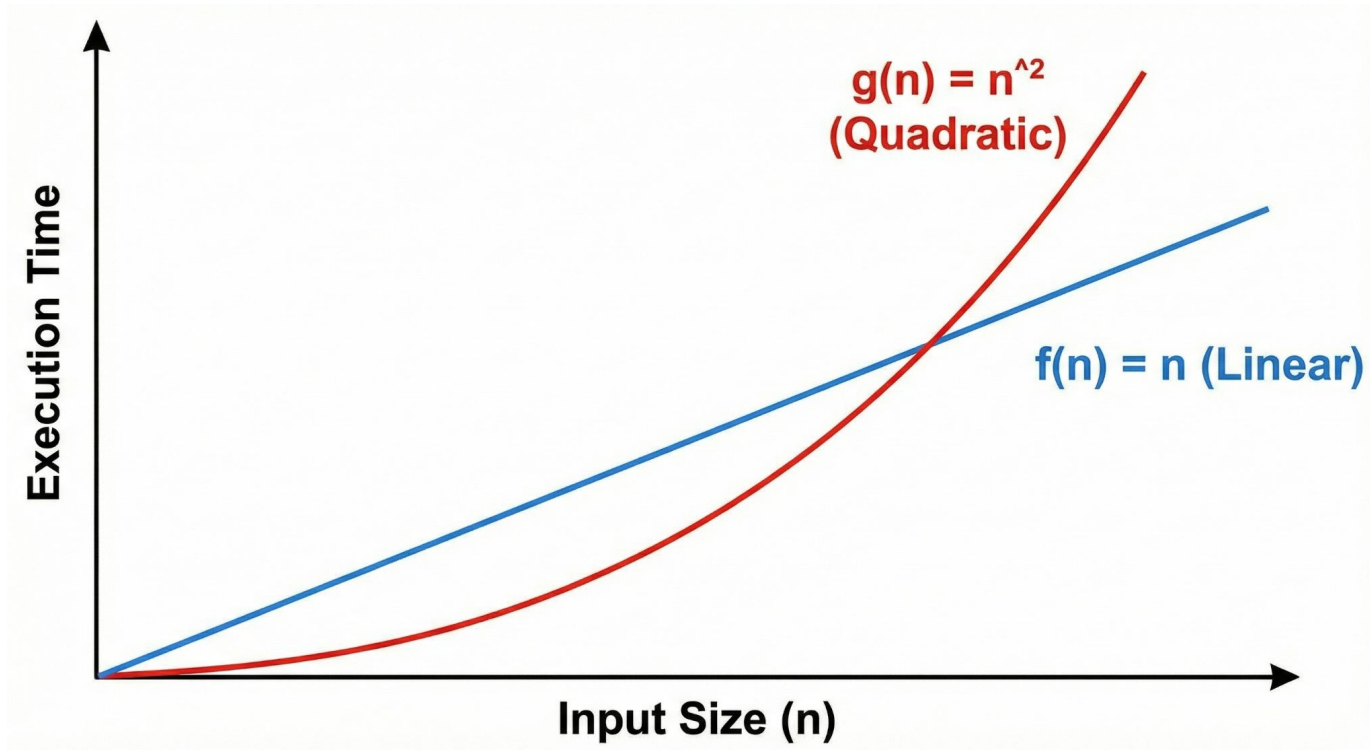


Big-O Notation

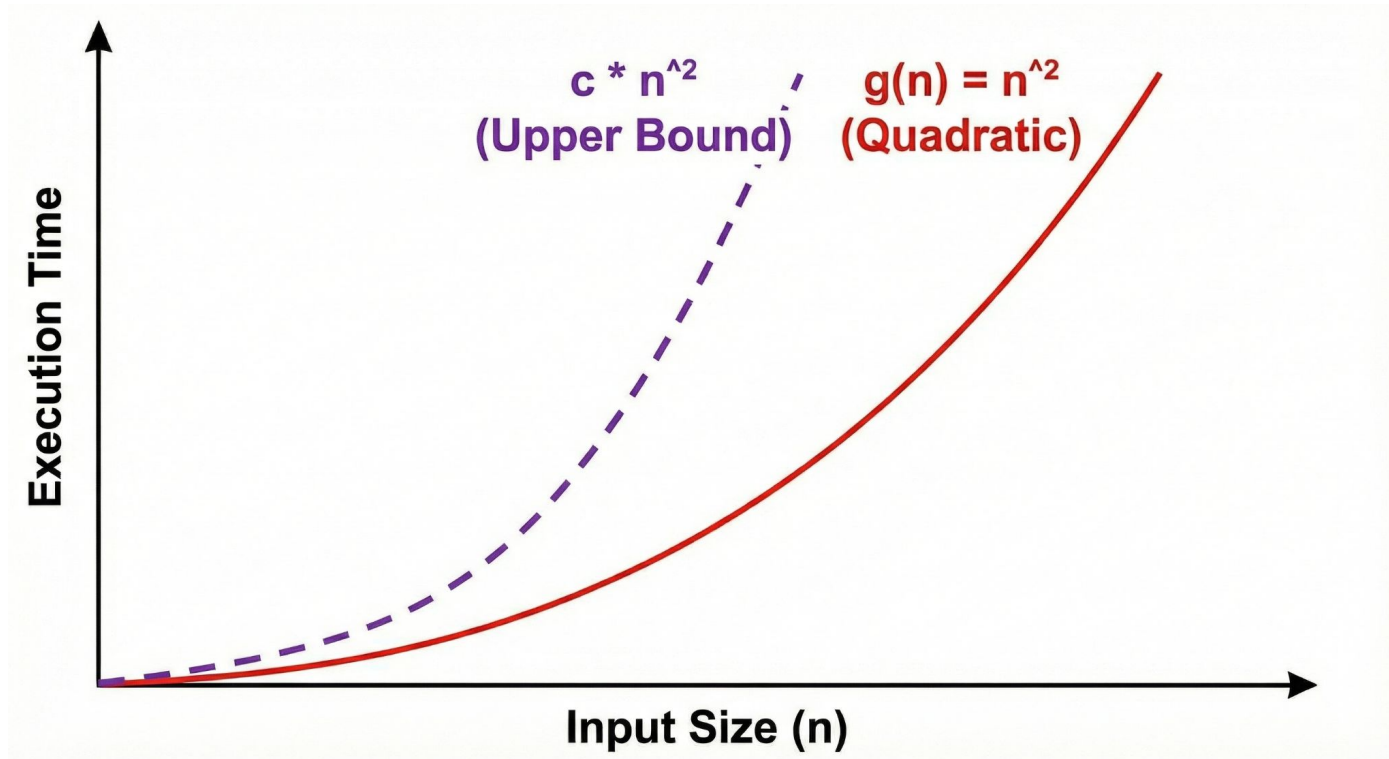


- Objectively describe the efficiency of code without the use of concrete units (seconds/bytes).
- Provide a big picture of how the time and space requirements scale w.r.t input size.
- Focus on **worst-case** scenario.

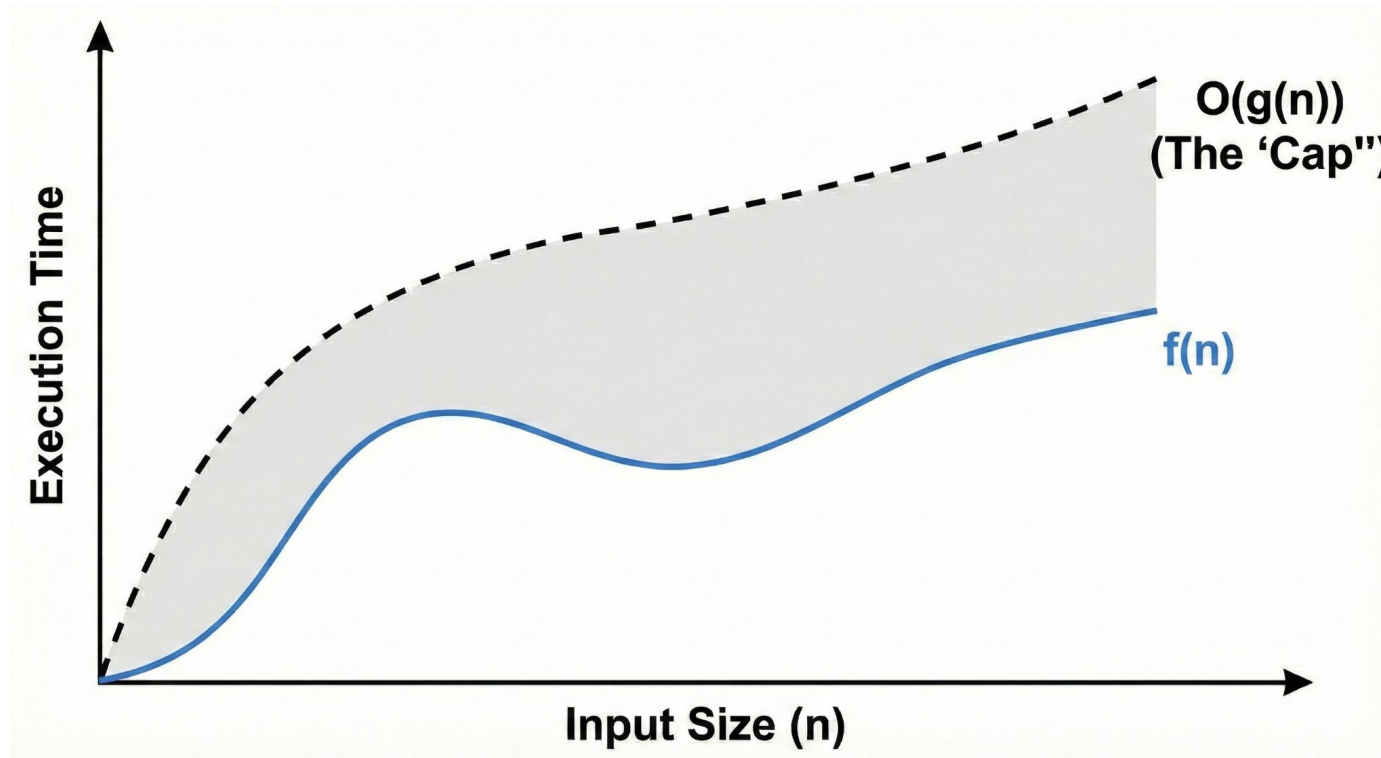
Big-O Notation



Big-O Notation



Big-O Notation





Big-O Notation



- Formally, $f(n) = O(g(n))$:
 - If there exist positive constants c and n_0
 - such that $0 < f(n) < c * g(n)$ for all $n \geq n_0$
- $f(n)$ is a big-O of $g(n)$
 - Intuitively means that g (multiplied by a constant factor) set an **upper bound** of f as n gets large - i.e., an **asymptotic bound**
 - It provides a theoretical **upper bound** on execution time.



Big-O Notation



Example: $f(n) = 4n^3 + 3n^2 + 5$

- Need to find $g(n)$, c , n_0 that satisfies $0 < f(n) < c * g(n)$ for all $n \geq n_0$



Big-O Notation



Example: $f(n) = 4n^3 + 3n^2 + 5$

- Need to find $g(n)$, c , n_0 that satisfies $0 < f(n) < c * g(n)$ for all $n \geq n_0$
- For example, $g(n) = n^3$, $c = 5$, $n_0 = 50$ satisfies, then
 - $f(n) = O(g(n)) = O(n^3)$
 - Big-O is an asymptotic upper bound or “worst case”
 - $f(n)$ will never goes beyond Big-O - $O(n^3)$



Simplifying Big-O



- Product Rule
 - If the Big-O is the product of the multiple terms, **drop the constant terms**

$$O(1024 * n) =$$

$$O(n / 10) =$$

$$O(7 * n * n) =$$

$$O(345) =$$



Simplifying Big-O



- Sum Rule
 - If the Big-O is the sum of the multiple terms, **only keep the largest term, drop the rest.**

$$O(100 + n) =$$

$$O(n^2 + n) =$$

$$O(n + 500 + n^3 + n^2) =$$



Big-O Notation



- Example:
 - $T(n) = 1 + n$,
then $T(n) =$
 - $T(n) = 5n^2 + 10n + 12$,
then $T(n) =$
- **$O(n^2)$ means time complexity $T(n)$ will never exceed $O(n^2)$.**



Big-O Notation (Ordered)



- Common $f(n)$:
 - $O(1)$: constant
 - $O(\log n)$: logarithm
 - $O(n)$: Linear
 - $O(n \log n)$: Log linear
 - $O(n^2)$: Quadratic
 - $O(2^n)$: Exponential
 - $O(n!)$: Factorial

| $O(1)$ - Constant Time

Task: Pick up the "Top paper of the Stack"

Imagine grabbing the top paper off a stack. It takes the same amount of effort whether the stack is 1 inch high or 1 mile high.

```
item = prices[0]
```

Key Takeaway: Instant access. Zero growth in time.



| $O(\log n)$ - Logarithmic Time

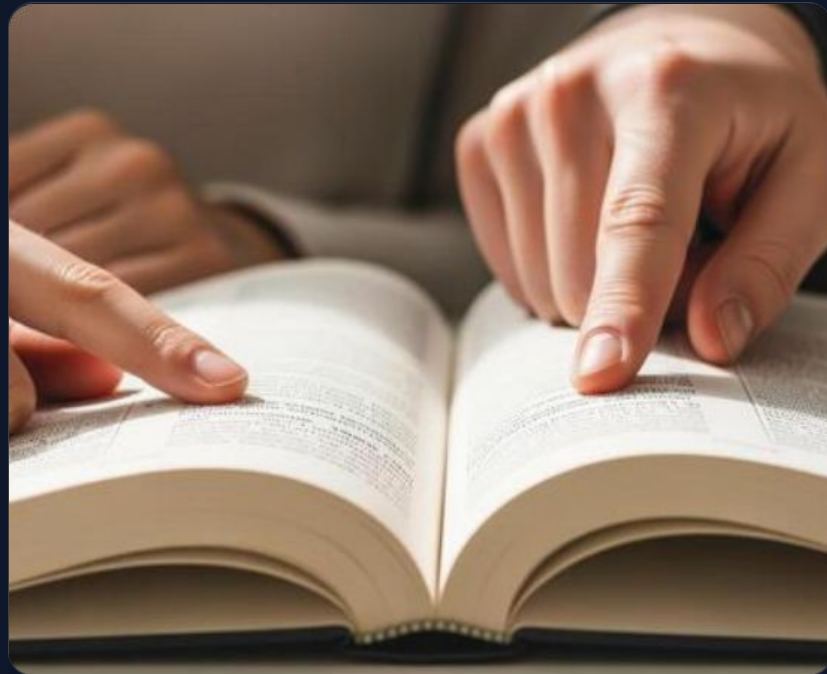
Task: Finding a page in a book

Imagine finding a page in a book.

You open the middle. If the page is too high, you ignore the second half. You keep splitting the problem in half.

```
# Binary Search  
while low <= high: ...
```

Key Takeaway: Extremely efficient. 1,000,000 items takes only ~20 steps.



O(n) - Linear Time

Task: The "Receipt Check"

Imagine reading a receipt to find a mistake.

You must check line 1, then line 2, then line 3,

all the way to the end. You cannot skip anything.

```
for item in prices:  
    print(item)
```

Key Takeaway: Fair growth. If data doubles, time doubles.

AD CNTRY SLC	3.49	F
D BEEF 30% 2LB	7.98	F
UL ONIONS 12PK	3.99	F
ATOS VALU 6CT	2.99	F
KDNY BEANS 34OZ	2.29	F
LCU ZSTY SPCE		
2 FOR 3.50	1.75	F
H GUAC	4.99	F
LMAN'S MAYO 30OZ	3.99	F
RAINE HRTS 4CT	3.49	F
RDEE TRASH BAGS	8.99	F
ITOS PRT SIZE	1.49	F
ES BK		
2 FOR 0.99	1.98	F
K CAPN CRUNCH	4.99	F
COUP PDN 12PK	5.49	F
ET MX CUP TROP 12CT	7.99	F
ROAST BEEF MED		
D LB @ 7.99/ 1b	11.99	F
DELI HAM NTFREE		
D LB @ 5.80/ 1b	11.60	F
CUITS BLK PEP		
2 FOR 4.50	2.25	F
L PAPER TOWELS 24CT	19.99	F
DRELLE DINR FMLY	10.99	F
CHURN VANILLA	5.49	F

O(n log n) - Linearithmic Time

Task: Card Sorting

Imagine sorting a deck of cards.

You have to touch every card (n), and you shuffle through them multiple times ($\log n$) to get them in order.

```
prices.sort()
```

Key Takeaway: Standard for sorting. Slower than $O(n)$, but necessary for ranking.



$O(n^2)$ - The Danger Zone

Task: The "Handshake Problem"

Imagine a networking event.

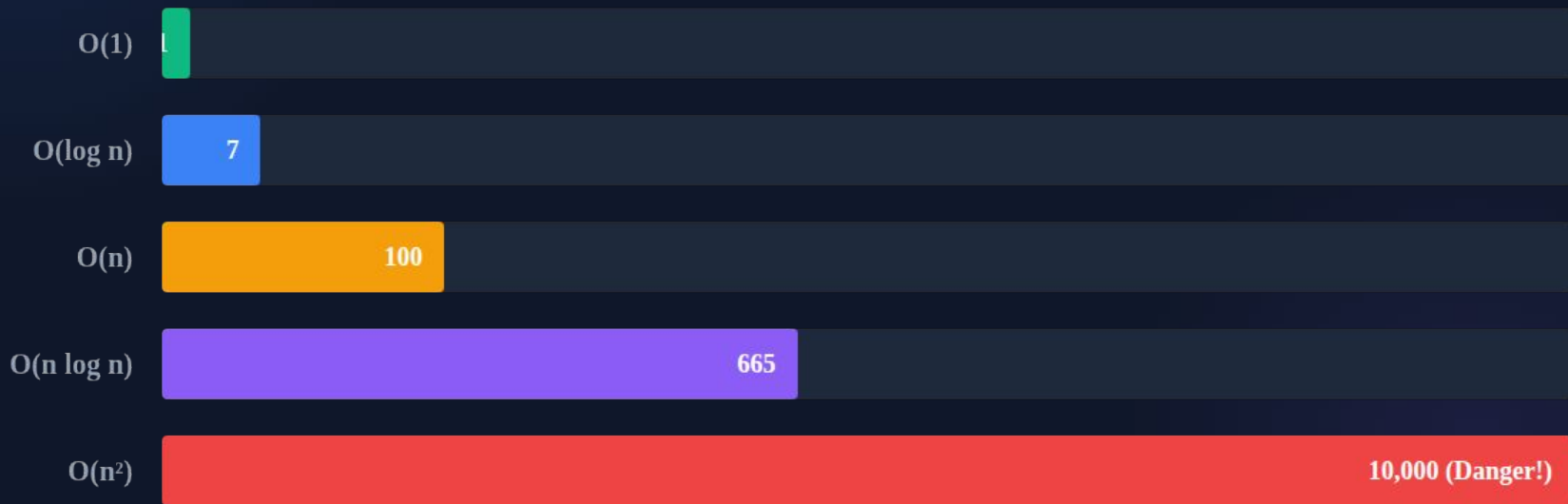
If everyone has to shake hands with everyone else, the number of handshakes explodes as more people arrive.

```
for x in list:  
    for y in list: ...
```

Warning: Doubling input size quadruples the run time. Avoid for large datasets.



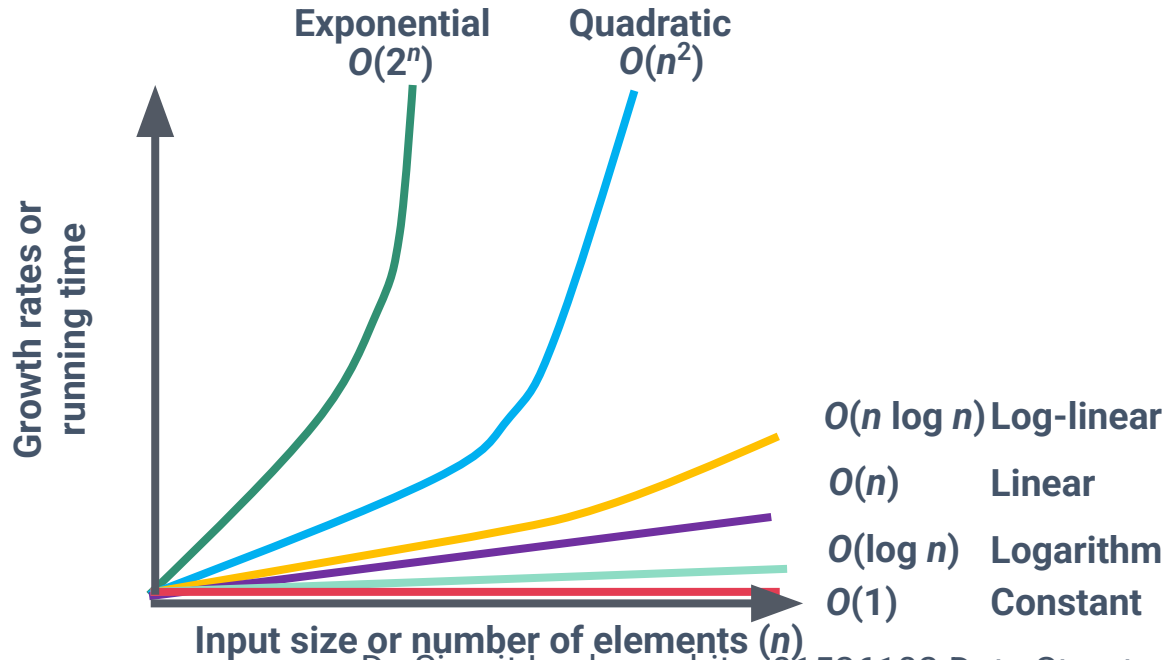
Scalability Summary (N=100)



Comparing the number of operations required for an input size of just 100 items.

Time Complexity

In general, **the standard functions** of input size n are shown in figure.





Algorithm Computational Complexity



1. Use **running time** as an indicator.
2. Use **number of operations or steps** as an indicator.
 - Represented by $T(n)$
3. Use asymptotic notation as an indicator.
 - **Big-O notation**